

**TRƯỜNG ĐẠI HỌC CÔNG NGHỆ
VIỆN TRÍ TUỆ NHÂN TẠO**



**BÁO CÁO MÔN HỌC
KỸ THUẬT VÀ CÔNG NGHỆ DỮ LIỆU LỚN CHO TRÍ
TUỆ NHÂN TẠO**

ĐỀ TÀI:

**Real-Time Network Monitoring
and Fault Detection**

Nhóm sinh viên thực hiện: 1. Nguyễn Đoàn Nhật Minh - 24022403
2. Phạm Lê Việt Đức - 24022296

Giảng viên hướng dẫn: TS. Trần Hồng Việt
ThS. Ngô Minh Hương

MỞ ĐẦU

Trong thời đại số hóa và sự bùng nổ của Internet, an ninh mạng đã trở thành một yếu tố sống còn đối với mọi tổ chức và doanh nghiệp. Sự gia tăng liên tục của các thiết bị kết nối IoT, hệ thống đám mây và lưu lượng truy cập tốc độ cao đã làm cho dữ liệu mạng gia tăng một cách chóng mặt cả về khối lượng lẫn tốc độ. Điều này dẫn đến sự ra đời của khái niệm dữ liệu lớn trong giám sát mạng. Các phương pháp phát hiện xâm nhập truyền thống (IDS) hoặc các hệ thống xử lý tập trung trên một máy chủ đơn lẻ đã bộc lộ những hạn chế nghiêm trọng: không thể mở rộng để xử lý lưu lượng mạng khổng lồ và gặp độ trễ lớn khi thực thi các dự báo thời gian thực.

Dự án này xây dựng một hệ thống giám sát lưu lượng mạng và phát hiện xâm nhập thời gian thực (IDS Real-Time Network Monitoring System) có khả năng mở rộng mạnh mẽ trên nền tảng công nghệ Big Data. Bằng cách kết hợp kiến trúc xử lý luồng phân tán và các mô hình học máy của PySpark MLlib, hệ thống có khả năng ingest và phân tích hàng nghìn luồng traffic mỗi giây trực tiếp từ các Broker của Apache Kafka. Dữ liệu sau khi xử lý sẽ được phân tích đặc trưng và đưa qua mô hình phân loại Random Forest Classifier chạy song song trực tiếp trên JVM của Spark để dự đoán lưu lượng là Benign (an toàn) hay Attack (xâm nhập độc hại).

Các thông số thống kê theo thời gian thực được đẩy trực tiếp vào cơ sở dữ liệu chuỗi thời gian InfluxDB và trực quan hóa sinh động trên Dashboard của Grafana. Khi phát hiện các dấu hiệu tấn công mạng, hệ thống tự động định tuyến các gói tin cảnh báo tới Kafka alerts topic và kích hoạt Telegram Alerter để gửi thông báo khẩn cấp tới quản trị viên. Toàn bộ hạ tầng hệ thống được container hóa thông qua Docker Compose, đảm bảo tính dễ dàng trong triển khai và vận hành ổn định trên nhiều môi trường khác nhau.

Mục lục

MỞ ĐẦU	i
1 TỔNG QUAN VỀ ĐỀ TÀI	1
1.1 Giới thiệu bài toán và tầm quan trọng	1
1.1.1 Tại sao việc giám sát mạng và phát hiện xâm nhập lại quan trọng?	1
1.1.2 Thách thức trong môi trường dữ liệu lớn	1
1.2 Mục tiêu cụ thể của dự án	1
1.3 Phạm vi và hạn chế của dự án	2
1.3.1 Phạm vi dự án	2
1.3.2 Hạn chế của dự án	2
2 PHƯƠNG PHÁP VÀ KIẾN TRÚC HỆ THỐNG	3
2.1 Các công nghệ sử dụng	3
2.2 Thu thập và tiền xử lý dữ liệu	4
2.2.1 Nguồn dữ liệu thực nghiệm	4
2.2.2 Quy trình tiền xử lý phục vụ huấn luyện mô hình	4
2.3 Mô hình hóa và dự báo phân tán	4
2.3.1 Xây dựng Pipeline học máy phân tán	4
2.3.2 Đánh giá mô hình và kết quả thực nghiệm	5
2.4 Kiến trúc triển khai và quy trình vận hành	5
2.4.1 Quy trình 4 lớp hoạt động của hệ thống	6
3 THỰC NGHIỆM VÀ KẾT QUẢ	8
3.1 Môi trường và cài đặt triển khai	8
3.2 Thực nghiệm	8
3.3 Kết quả đạt được	9
3.3.1 Giao diện giám sát mạng (Grafana Dashboard)	9
3.3.2 Hệ thống cảnh báo (Telegram Alerting)	10
4 KẾT LUẬN VÀ HƯỚNG PHÁT TRIỂN	11
4.1 Kết luận và Những thành tựu đạt được	11
4.1.1 Tóm tắt thành tựu	11
4.1.2 Ý nghĩa thực tiễn	11
4.1.3 Thách thức và hạn chế	11
4.2 Hướng phát triển trong tương lai	12
PHÂN CÔNG CÔNG VIỆC	13

Chương 1

TỔNG QUAN VỀ ĐỀ TÀI

1.1 Giới thiệu bài toán và tầm quan trọng

1.1.1 Tại sao việc giám sát mạng và phát hiện xâm nhập lại quan trọng?

Hệ thống phát hiện xâm nhập mạng (NIDS - Network Intrusion Detection System) đóng vai trò như một lớp phòng thủ chủ động bằng cách liên tục giám sát lưu lượng mạng để tìm kiếm các hành vi bất thường hoặc dấu hiệu xâm nhập độc hại (như DoS, DDoS, Botnet, Portscan, Web Attacks). Việc phát hiện sớm các cuộc tấn công này giúp ngăn chặn kịp thời các rủi ro liên quan đến rò rỉ dữ liệu nhạy cảm, phá hoại hạ tầng công nghệ thông tin và tổn thất tài chính cho tổ chức.

1.1.2 Thách thức trong môi trường dữ liệu lớn

Hệ thống NIDS hiện đại phải đối mặt với các đặc trưng cơ bản của Big Data bao gồm:

- **Volume:** Hàng triệu kết nối mạng được thiết lập mỗi phút, tạo ra lượng log khổng lồ cần được lưu trữ và truy vấn hiệu quả.
- **Velocity:** Luồng dữ liệu mạng truyền đi với tốc độ gigabite/giây đòi hỏi hệ thống xử lý phải có độ trễ cực thấp (dưới vài giây) từ khi nhận dữ liệu đến khi đưa ra cảnh báo.
- **Variety:** Sự kết hợp của nhiều giao thức mạng khác nhau (TCP, UDP, ICMP) và các định dạng log đa dạng đòi hỏi hệ thống có cơ chế trích xuất đặc trưng linh hoạt.

1.2 Mục tiêu cụ thể của dự án

Dự án hướng tới việc thiết kế và hiện thực hóa một hệ thống IDS thời gian thực với các mục tiêu cụ thể sau:

- **Tự động hóa hoàn toàn quy trình xử lý:** Xây dựng một luồng dữ liệu tự động từ khâu giả lập sinh traffic mạng, phân phối qua Message Broker, phân tích bằng Spark Streaming, lưu trữ vào Database và kích hoạt Alerting.
- **Tối ưu hóa hiệu năng dự đoán:** Sử dụng mô hình học máy phân tán PySpark MLlib chạy trực tiếp trong JVM để dự đoán phân loại nhãn mạng thay vì các mô hình chạy ngoài driver (như Scikit-Learn thông qua PyArrow) để tránh nghẽn cổ chai bộ nhớ và tối ưu hóa tài nguyên tính toán.
- **Giám sát trực quan và tức thời:** Cung cấp dashboard Grafana trực quan hóa trực tiếp lưu lượng mạng, tỷ lệ tấn công và thông số TCP flags đồng thời tích hợp bot cảnh báo tự động qua ứng dụng Telegram.

1.3 Phạm vi và hạn chế của dự án

1.3.1 Phạm vi dự án

- Hệ thống tập trung phân loại dữ liệu lưu lượng mạng nhị phân thành hai nhóm chính: Benign (an toàn) và Attack (xâm nhập/tấn công chung).
- Sử dụng bộ dữ liệu CIC-IDS2017 chuẩn hóa gồm 76 đặc trưng lưu lượng mạng được trích xuất từ định dạng Parquet.
- Giả lập truyền nhận dữ liệu real-time thông qua Kafka Producer.

1.3.2 Hạn chế của dự án

- Chưa đi sâu vào phân loại chi tiết từng kiểu tấn công cụ thể (như phân tách rõ rệt giữa DDoS và SQL Injection).
- Quá trình huấn luyện mô hình được thực hiện offline trên một tập mẫu đại diện đại diện 10% của toàn bộ dataset để tránh lỗi tràn bộ nhớ của máy ảo WSL cá nhân.
- Chưa có module tích hợp nguồn dữ liệu thực tế

Chương 2

PHƯƠNG PHÁP VÀ KIẾN TRÚC HỆ THỐNG

2.1 Các công nghệ sử dụng

Hệ thống được phát triển và tích hợp trên nền tảng các công nghệ dữ liệu lớn phân tán hiện đại, bao gồm:

- **Apache Kafka và Zookeeper:** Đóng vai trò là hệ thống Ingestion Layer. Kafka hoạt động như một hàng đợi tin nhắn phân tán thu nhận dữ liệu mạng thô (network flows) từ client và phân phối đến Spark Streaming.
- **Apache Spark và Spark Streaming:** Đóng vai trò là Stream Processing Layer. Sử dụng kiến trúc micro-batch của Structured Streaming để đọc dữ liệu từ Kafka, áp dụng mô hình học máy để dự báo nhân và tính toán thống kê theo từng batch.
- **PySpark MLlib:** Thư viện học máy phân tán của Apache Spark. Dự án sử dụng mô hình học máy PipelineModel (chứa RandomForestClassifier) để thực hiện dự đoán phân tán trực tiếp trên Spark Executor JVM.
- **InfluxDB:** Cơ sở dữ liệu chuỗi thời gian được tối ưu hóa để lưu trữ các thông số giám sát mạng tính toán từ Spark Streaming nhằm mục đích hiển thị biểu đồ thời gian thực.
- **Grafana:** Nền tảng hiển thị và phân tích dữ liệu. Cung cấp giao diện giám sát trực quan kết nối trực tiếp với InfluxDB.
- **Docker:** Đóng gói toàn bộ các dịch vụ (Zookeeper, Kafka, InfluxDB, Grafana) dưới dạng Container giúp hệ thống dễ dàng triển khai và độc lập với môi trường máy chủ.

2.2 Thu thập và tiền xử lý dữ liệu

2.2.1 Nguồn dữ liệu thực nghiệm

Dữ liệu đầu vào của hệ thống được trích xuất từ bộ dữ liệu giám sát an ninh mạng tiêu chuẩn **CIC-IDS2017** (của Đại học UNB, Canada). Quy trình thu thập diễn ra như sau:

- **Tải dữ liệu thô:** Tải trực tiếp các tệp dữ liệu gốc định dạng .csv từ trang cung cấp của CIC-IDS2017. Các tệp này ghi nhận chi tiết thống kê luồng mạng theo từng ngày trong tuần.
- **Tiền xử lý sơ bộ:** Do các tệp CSV gốc có dung lượng rất lớn và tốc độ đọc/ghi kém tối ưu trong môi trường phân tán, toàn bộ dữ liệu CSV được chạy qua các notebook tiền xử lý sơ bộ để chuyển đổi và lưu trữ dưới định dạng **Parquet**.

2.2.2 Quy trình tiền xử lý phục vụ huấn luyện mô hình

Trong pha huấn luyện ngoại tuyến (offline training) với Apache Spark, dữ liệu từ các tệp Parquet (hơn 2.3 triệu mẫu) tiếp tục được làm sạch và tối ưu qua các bước:

1. **Nhãn hóa mục tiêu:** Ánh xạ cột Label sang nhãn số nhị phân target: nhãn mạng bình thường (Benign) tương ứng với 0, và tất cả các nhãn tấn công khác tương ứng với 1.
2. **Loại bỏ giá trị lỗi:** Lọc bỏ các dòng chứa giá trị inf, -inf hoặc NaN.
3. **Ép kiểu và điền giá trị khuyết thiếu:** Đảm bảo tất cả các đặc trưng số học được ép kiểu về dạng DoubleType. Bất kỳ ô trống (null) nào sót lại sẽ được điền giá trị 0.
4. **Staging dữ liệu trung gian:** Do việc kết hợp nhiều tệp Parquet và thực thi hàng loạt lệnh biến đổi sẽ làm phình to đồ thị luồng của Spark gây lỗi tràn bộ nhớ (OOM). Do đó, dữ liệu sau khi làm sạch được ghi tạm xuống đĩa bằng lệnh `coalesce(2).write.parquet()` nhằm ngắt liên kết lịch sử tính toán và dọn dẹp bộ nhớ đệm trước khi đẩy vào mô hình phân loại.

2.3 Mô hình hóa và dự báo phân tán

2.3.1 Xây dựng Pipeline học máy phân tán

Để đưa hệ thống vào vận hành trực tuyến một cách tối ưu, mô hình học máy được cấu trúc thành một Pipeline gồm ba giai đoạn:

1. **VectorAssembler:** Tập hợp toàn bộ 76 thuộc tính mạng thô thành một cột vector đặc trưng duy nhất (features).

2. **StandardScaler**: Chuẩn hóa các vector đặc trưng này để loại bỏ sự chênh lệch lớn về biên độ giữa các thuộc tính (như kích thước gói tin và thời gian trễ của dòng chảy), sinh ra cột đặc trưng chuẩn hóa (scaledFeatures).
3. **RandomForestClassifier**: Bộ phân loại Rừng ngẫu nhiên phân tán. Mô hình được tinh chỉnh để chạy tối ưu trên tài nguyên local với thông số: numTrees=20 (20 cây quyết định) và maxDepth=8 (độ sâu tối đa của cây là 8) nhằm cân bằng giữa độ chính xác và tốc độ dự đoán.

2.3.2 Đánh giá mô hình và kết quả thực nghiệm

Mô hình huấn luyện trên 10% mẫu dữ liệu được lưu trữ trực tiếp dưới dạng Spark PipelineModel tại ./training_model/spark_rf_model. Kết quả đánh giá trên tập kiểm thử thu được hiệu năng xuất sắc:

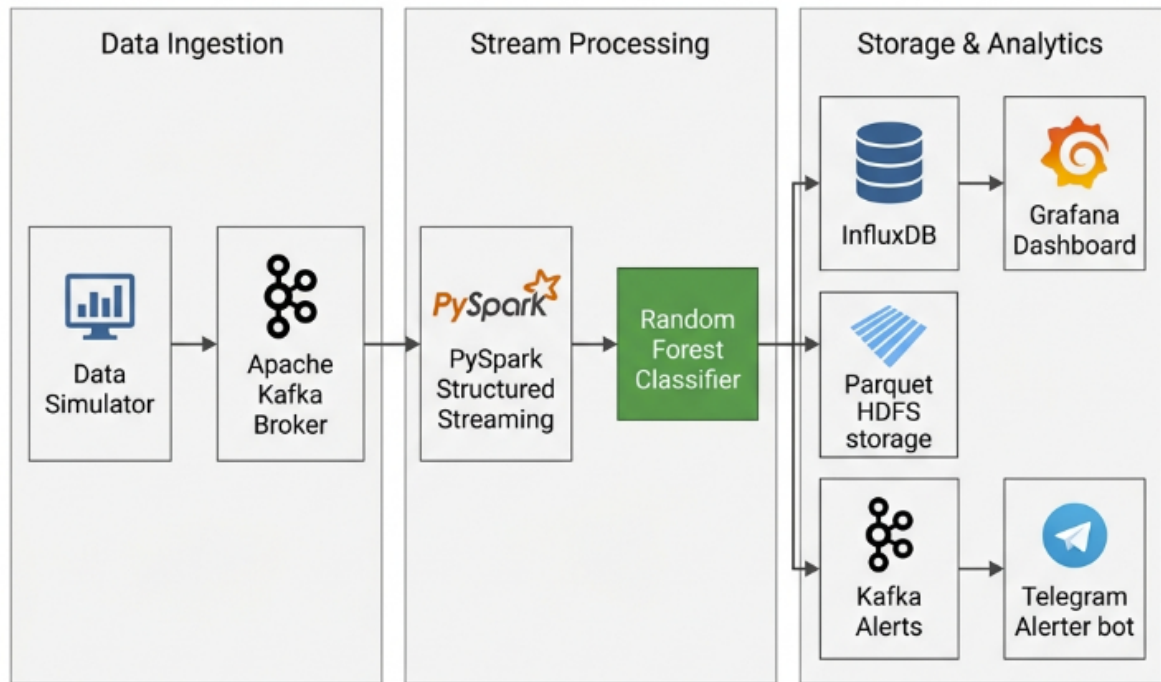
- Độ chính xác (Test Accuracy): **99.70%**
- Chỉ số F1-Score: **99.70%**

Việc lưu trữ trực tiếp dưới dạng PipelineModel giúp Spark Streaming có thể nạp mô hình gốc này tại thời điểm khởi động và thực hiện phân loại song song (model.transform(df)) trực tiếp trên Executor JVM, mang lại các lợi ích vượt trội so với việc gọi mô hình bên ngoài như Scikit-Learn thông qua Pandas UDF:

- Tránh hoàn toàn độ trễ của cơ chế tuần tự hóa và giải tuần tự hóa dữ liệu qua PyArrow giữa tiến trình Python và JVM.
- Giảm tải cực lớn cho Spark Driver node bằng cách đẩy các tác vụ dự đoán trực tiếp xuống các Executor Worker.

2.4 Kiến trúc triển khai và quy trình vận hành

Kiến trúc luồng xử lý thời gian thực của hệ thống tuân theo mô hình Lambda Architecture tối giản, được thể hiện thông qua quy trình 4 lớp:



Hình 2.1: Sơ đồ kiến trúc hệ thống

2.4.1 Quy trình 4 lớp hoạt động của hệ thống:

1. **Ingestion Layer (Thu thập):** Script `kafka_producer.py` giả lập hành vi của các thiết bị giám sát mạng bằng cách đọc các dòng traffic mạng từ Parquet dataset (bỏ nhãn thô), chuyển đổi sang JSON và gửi vào Kafka topic `network_traffic` với độ trễ 50ms cho mỗi gói tin.
2. **Stream Processing Layer (Xử lý luồng):** Script `spark_streaming.py` liên tục kéo dữ liệu từ topic `network_traffic`, nạp mô hình `PipelineModel` từ ổ đĩa và áp dụng hàm `model.transform()` để dự đoán nhãn thời gian thực.
3. **Storage Layer (Lưu trữ):**
 - Dữ liệu thô kèm nhãn dự đoán được ghi nối tiếp dưới định dạng Parquet vào thư mục lưu trữ cục bộ để phục vụ huấn luyện lại hoặc phân tích lịch sử.
 - Các chỉ số thống kê tổng hợp (tổng luồng mạng, số lượng tấn công, số lượng cờ TCP, tốc độ gói tin và byte trung bình) được ghi trực tiếp vào InfluxDB `ids_db` qua mỗi micro-batch.
4. **Serving và Alerting Layer (Cảnh báo và Giám sát):**
 - Grafana liên tục kết nối với InfluxDB để cập nhật dashboard giám sát thời gian thực.

- Nếu một micro-batch phát hiện có lưu lượng độc hại (`attack_count > 0`), Spark Streaming sẽ gửi một payload JSON chứa các thông số tổng hợp của batch đó vào Kafka topic `network_alerts`.
- Tiến trình `telegram_alerter.py` lắng nghe topic `network_alerts` và gửi tin nhắn cảnh báo bảo mật định dạng Markdown trực tiếp tới Telegram Chat ID của quản trị viên hệ thống.

Chương 3

THỰC NGHIỆM VÀ KẾT QUẢ

3.1 Môi trường và cài đặt triển khai

Hệ thống được thử nghiệm trên môi trường máy tính cá nhân và github codespace:

- **Hệ điều hành:** Windows Subsystem for Linux (WSL 2) - Ubuntu.
- **Ngôn ngữ và Nền tảng:** Python 3.8, Java 17 (dành cho JVM của Spark).
- Sử dụng Docker & Docker Compose để đóng gói khởi chạy Zookeeper, Kafka, InfluxDB và Grafana.
- **Thư viện chính:** PySpark, PyArrow, Kafka-Python.

3.2 Thực nghiệm

Để đánh giá khả năng xử lý thời gian thực của hệ thống, nhóm tiến hành một kịch bản thực nghiệm mô phỏng tấn công từ chối dịch vụ (DoS) như sau:

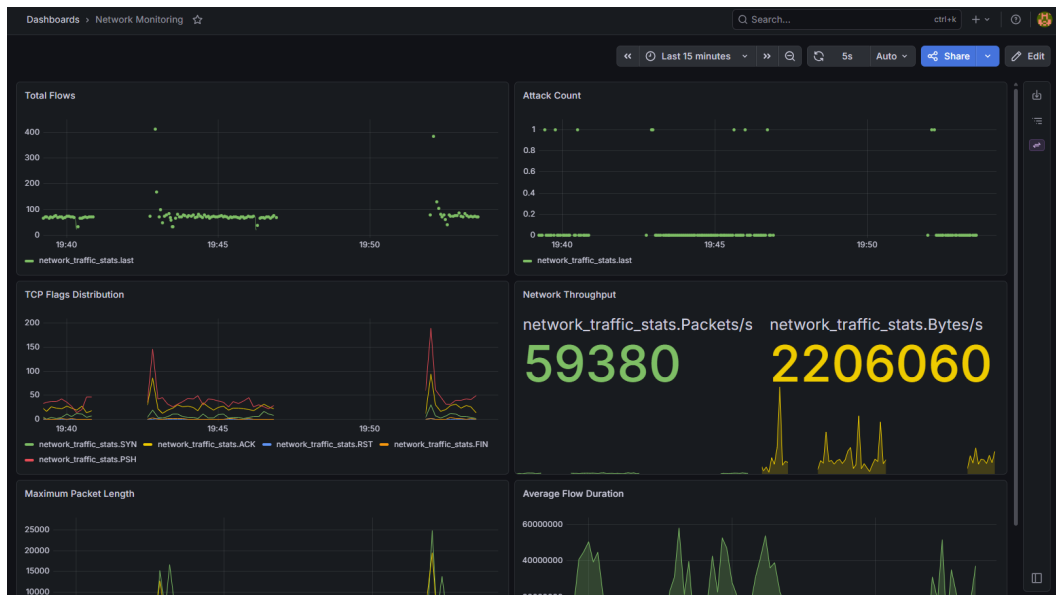
1. Khởi động hệ sinh thái nền tảng gồm Kafka, InfluxDB và Grafana bằng lệnh `docker-compose up`.
2. Khởi động mô-đun Spark Streaming (`spark_streaming.py`) và dịch vụ cảnh báo Telegram (`telegram_alerter.py`) để sẵn sàng tiếp nhận và xử lý dữ liệu.
3. Sử dụng tập dữ liệu `DoS-Wednesday-WorkingHours.pcap_ISCX.parquet` làm nguồn dữ liệu đầu vào cho chương trình `kafka_producer.py`, nhằm mô phỏng lượng lớn lưu lượng truy cập bất thường đổ về hệ thống trong một cuộc tấn công DoS.

3.3 Kết quả đạt được

3.3.1 Giao diện giám sát mạng (Grafana Dashboard)

Dữ liệu mạng được tính toán mượt mà từ Spark qua InfluxDB và hiển thị tức thời trên Grafana với độ trễ 5 giây. Dashboard hiển thị trực quan các thông số bao gồm:

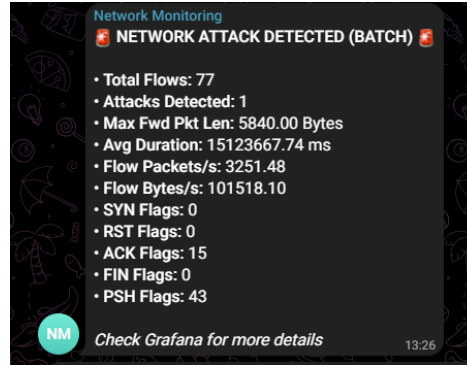
- **Total Flows:** Tổng số lượng luồng mạng (flows) được ghi nhận trong hệ thống.
- **Attack Count:** Tổng số lượng các gói tin bị mô hình đánh nhãn là Attack.
- **TCP Flags Distribution:** Biểu đồ phân phối tỷ lệ các cờ TCP (như SYN, ACK, FIN) giúp quản trị viên xác minh loại hình tấn công (ví dụ: SYN Flood).
- **Network Throughput:** Bảng thông và lưu lượng truyền tải thực tế của hệ thống mạng.
- **Maximum Packet Length:** Kích thước lớn nhất của gói tin, giúp nhận diện sự bất thường về payload so với traffic thông thường.
- **Average Flow Duration:** Thời gian duy trì kết nối trung bình của các luồng, đặc biệt quan trọng để phát hiện các cuộc tấn công ngốn tài nguyên kéo dài.



Hình 3.1: Giao diện giám sát lưu lượng mạng theo thời gian thực trên Grafana.

3.3.2 Hệ thống cảnh báo (Telegram Alerting)

Ngay khi biểu đồ tần công tăng vọt, luồng Spark Streaming tự động phát hiện, gom nhóm cảnh báo và bắn thông điệp sang Kafka. Tiến trình Alerter bắt được thông điệp và lập tức đẩy notification về Telegram của quản trị viên.



Hình 3.2: Tin nhắn cảnh báo được gửi tự động về Telegram quản trị viên.

Chương 4

KẾT LUẬN VÀ HƯỞNG PHÁT TRIỂN

4.1 Kết luận và Những thành tựu đạt được

4.1.1 Tóm tắt thành tựu

Dự án đã xây dựng thành công một hệ thống phát hiện xâm nhập mạng thời gian thực hoạt động ổn định và có hiệu năng cao trên nền tảng Big Data. Các kết quả cụ thể bao gồm:

- Triển khai thành công mô hình Random Forest Classifier phân tán thông qua thư viện PySpark MLlib cho kết quả chính xác cao đạt **99.70%** trên tập dữ liệu kiểm thử.
- Tối ưu hóa thành công quá trình nạp và dự đoán của mô hình trực tiếp trong JVM, giúp hệ thống đạt tốc độ xử lý hàng trăm bản ghi mạng mỗi giây với độ trễ tối thiểu (dưới 1 giây).
- Tích hợp cơ chế đồng bộ hóa dữ liệu từ Spark Streaming qua InfluxDB đến Grafana và Telegram tự động, tạo nên một chu trình giám sát an ninh mạng khép kín và an toàn.

4.1.2 Ý nghĩa thực tiễn

Hệ thống cung cấp cho các quản trị viên mạng một công cụ giám sát trực quan, giúp nhanh chóng phát hiện các dấu hiệu tấn công bất thường (như quét cổng hàng loạt hoặc tấn công từ chối dịch vụ thông qua lượng cờ SYN tăng vọt) để đưa ra các biện pháp phản ứng kịp thời thông qua cảnh báo tức thì từ ứng dụng Telegram.

4.1.3 Thách thức và hạn chế

- Do chạy thử nghiệm trên máy tính cá nhân qua môi trường WSL, việc huấn luyện mô hình phân tán quy mô lớn vẫn bị hạn chế về mặt RAM và

luồng xử lý.

- Việc cấu hình các đường dẫn file cục bộ trong Spark yêu cầu thiết lập thuộc tính hệ thống tệp tin phù hợp để tránh lỗi xung đột lớp nạp tập tin nội bộ của Hadoop.

4.2 Hướng phát triển trong tương lai

- **Mở rộng mô hình phân loại:** Nâng cấp bộ phân loại từ nhị phân sang đa lớp để xác định chính xác tên loại hình tấn công mạng cụ thể (như Bruteforce, Infiltration, Botnet).
- **Tích hợp các mô hình học sâu:** Khảo sát và triển khai các mô hình học sâu chuỗi thời gian (như LSTM, GRU) trên Spark thông qua các framework phân tán nâng cao (như TensorFlow on Spark) để cải thiện độ chính xác dự báo.
- **Tự động hóa hành vi ứng phó (Auto-healing):** Phát triển cơ chế tự động khóa IP hoặc ngắt kết nối mạng tạm thời thông qua API tường lửa của hệ thống ngay khi Telegram Alerter kích hoạt cảnh báo xâm nhập.

PHÂN CÔNG CÔNG VIỆC

Để hoàn thành dự án một cách tốt nhất, công việc được phân chia công bằng và phù hợp với vai trò của từng thành viên trong nhóm như sau:

Bảng 4.1: Bảng phân công nhiệm vụ và vai trò thành viên.

Thành viên	Nhiệm vụ cụ thể
Phạm Lê Việt Đức	- Huấn luyện và đánh giá mô hình Random Forest Classifier phân tán offline trên Jupyter Notebook. - Xây dựng luồng xử lý dữ liệu thời gian thực sử dụng PySpark Structured Streaming. - Tích hợp mô hình học máy dạng PipelineModel chạy trực tiếp trên JVM của Spark. - Tối ưu hệ thống và viết báo cáo.
Nguyễn Đoàn Nhật Minh	- Thiết kế và triển khai lớp thu thập dữ liệu sử dụng Apache Kafka và Zookeeper. - Thiết lập cơ sở dữ liệu chuỗi thời gian InfluxDB và xây dựng Dashboard trực quan hóa trên Grafana. - Lập trình module tích hợp cảnh báo bảo mật thời gian thực thông qua Telegram Bot API. - Container hóa toàn bộ hệ thống bằng Docker Compose.